

I 1 Đại số tuyến tính ứng dụng (Applied Linear Algebra)

PHỤ THUỘC không — đây là mục mở đầu của cây. Nếu bạn đã cài hiệu chỉnh chùm tia (*bundle adjustment*) thì phần không gian con là ôn lại; chỗ đáng đọc kỹ là *số điều kiện* và *hạng hiệu dụng*, hai chỗ mà ngôn ngữ SLAM và ngôn ngữ deep learning gọi cùng một hiện tượng bằng hai cái tên khác nhau.

TÓM TẮT

Mục này rút đại số tuyến tính xuống đúng bốn thứ quay lại suốt cây: không gian con và phép chiếu, hai phân rã eigen và SVD, số điều kiện, xấp xỉ hạng thấp. Đọc xong bạn phân biệt được “huấn luyện chậm vì bài toán điều hoà tẻ” với “chậm vì learning rate sai”. Bạn đọc được phổ trị kỳ dị của một tầng đặc trưng. Và bạn biết vì sao một solver nghiêm túc không bao giờ lập $A^T A$. Toàn bộ được neo vào thứ bạn đã làm được: hiệu chỉnh chùm tia, ellipsoid hiệp phương sai, gauge freedom. Nếu chỉ nhớ một điều: bình phương tối thiểu là một phép chiếu vuông góc, còn số điều kiện là hệ số khuếch đại sai số của *bài toán* chứ không phải của thuật toán. Hệ quả: đối optimizer không cứu được một bài toán bị điều hoà tẻ.

KIẾN THỨC CẦN CÓ TRƯỚC

- **Quy ước ký hiệu** — vector luôn là vector *cột*; A^T là ma trận chuyển vị; ma trận $m \times n$ có m hàng và n cột; $\|\cdot\|$ không ghi chỉ số nghĩa là chuẩn Euclid (chuẩn 2).
- **mô hình (model)** — một hàm có tham số, nhận đầu vào và trả về dự đoán. “Huấn luyện” nghĩa là đi tìm bộ tham số làm dự đoán sát nhất.
- **trọng số (weight)** — chính là các tham số của mô hình. Trong một tầng tuyến tính, chúng được xếp thành ma trận W và phép tính là $Wx + b$.
- **gradient** — vector gồm các đạo hàm riêng của một hàm theo từng tham số; nó chỉ hướng làm hàm tăng nhanh nhất, nên thuật toán tối ưu đi *ngược* hướng đó.
- **ma trận Hessian** — ma trận đạo hàm bậc hai; nó đo *độ cong* của hàm — hướng nào dốc gắt, hướng nào phẳng.
- **trị riêng và vector riêng** — vector riêng là hướng mà ma trận chỉ kéo dãn chứ không xoay; trị riêng là hệ số kéo dãn theo hướng đó. Tập hợp các trị riêng gọi là *phổ* của ma trận.
- **hạng (rank)** — số chiều thật sự mà đầu ra của ma trận trải ra. Hạng thấp nghĩa là ma trận nén dữ liệu về một không gian con nhỏ hơn.
- **hiệu chỉnh chùm tia (bundle adjustment)** — trong SLAM và dựng lại 3D, bài toán tinh chỉnh đồng thời mọi tư thế camera và mọi điểm 3D bằng cách cực tiểu tổng bình phương sai số tái chiếu. Nó là ví dụ chuẩn của bình phương tối thiểu phi tuyến quy mô lớn.

Đọc nhanh. Nếu bốn ý trên đã rõ, đọc thẳng tiểu mục D (số điều kiện) và E (giới hạn).

A. Đặt vấn đề

Một kỹ sư SLAM giải hệ tuyến tính mỗi ngày mà không gọi tên nó là đại số tuyến tính: mỗi vòng *Gauss–Newton* của hiệu chỉnh chùm tia là một bài bình phương tối thiểu. Mỗi ma trận **hiệp phương sai** (*covariance*) của keyframe là một dạng toàn phương đối xứng. Và mỗi lần khôi phục pose từ ma trận cơ bản là một lần gọi **phân rã giá trị kỳ dị** (*singular value decomposition*, SVD). Khi bước sang deep learning, những đối tượng ấy quay lại nguyên vẹn nhưng dưới tên khác: “không gian đặc trưng” thay cho không gian cột, *rank collapse* thay cho ma trận suy biến, “*ill-conditioned loss*” thay cho số điều kiện lớn. Vấn đề không phải thiếu kiến thức mà thiếu ánh xạ giữa hai bộ từ vựng.

Học xong mục này thì làm được gì mà trước đó không làm được? Ba việc rất cụ thể: (i) nhìn một đường cong huấn luyện chậm và phân biệt được “bài toán bị điều hoà tẻ” với “learning rate

sai” với “mô hình quá nhỏ”; (ii) đọc một biểu đồ phổ trị kỳ dị của tầng đặc trưng và nói được mô hình đang thu hẹp biểu diễn hay không; (iii) biết vì sao một thư viện giải bài bình phương tối thiểu nghiêm túc không bao giờ lập ma trận $A^T A$.

Mục này không dạy lại đại số tuyến tính. Nó chọn ra bốn ý sẽ quay lại suốt Phần B, C, D và cho mỗi ý một chỗ neo trong thứ bạn đã làm được bằng C++:

• **Bốn ý được giữ lại, và câu hỏi mà mỗi ý trả lời:**

- **không gian con** (*subspace*) và **phép chiếu** (*projection*) — “nghiệm bình phương tối thiểu là cái gì, chứ không phải tính bằng công thức nào”.
- **phân rã trị riêng** (*eigendecomposition*) và SVD — “ma trận này kéo dãn không gian theo những trục nào, mạnh yếu ra sao”.
- **số điều kiện** (*condition number*) — “tôi sắp mất bao nhiêu chữ số, và hướng nào của bài toán gần như không bị ràng buộc”.
- **xấp xỉ hạng thấp** (*low-rank approximation*) — “giữ lại bao nhiêu là đủ, và bỏ đi cái gì”.

Nguyên tắc của cả Phần A áp dụng nguyên: chỉ tới mức dùng được và giải thích được, không chứng minh định lý. Ai muốn chứng minh thì đọc [1]; bản trình bày định hướng học máy ở [2].

TRỰC GIÁC

Bạn đã nhìn thấy SVD mỗi lần vẽ ellipsoid bất định của một keyframe mà không biết. Ellipsoid đó là hình ảnh của quả cầu đơn vị sau khi bị ma trận hiệp phương sai kéo dãn: hướng các trục chính là các **vector riêng** (*eigenvector*), độ dài các trục là căn của các **trị riêng** (*eigenvalue*). Mọi ma trận đều làm đúng ba động tác ấy với quả cầu đơn vị — xoay, kéo dãn theo vài trục ưu tiên, rồi xoay lần nữa — và SVD chỉ là biên bản ghi lại ba động tác. Khi ellipsoid dẹt gần thành đĩa, bài toán suy biến theo một hướng, và đó chính là lúc số điều kiện lớn.

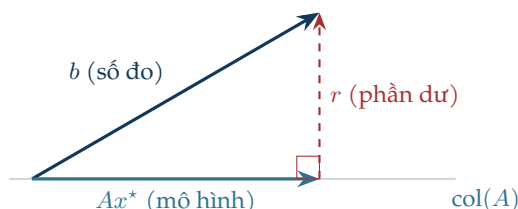
B. Không gian con và phép chiếu: bình phương tối thiểu không phải một công thức

Bình phương tối thiểu (*least squares*) thường được nhớ như công thức $x^* = (A^T A)^{-1} A^T b$. Nhớ như vậy thì không giải thích được vì sao nó hoạt động và hỏng khi nào. Cách nhớ dùng được lâu hơn là hình học: cột của A căng ra một không gian con của $\mathbb{R}^m$; vector đo b nói chung không nằm trong đó vì có nhiều; nghiệm bình phương tối thiểu là *hình chiếu vuông góc* của b xuống không gian cột, và phần dư $r = b - Ax^*$ vuông góc với mọi cột của A . Điều kiện vuông góc viết ra là

$$A^T r = 0 \iff A^T A x = A^T b.$$

Nói bằng lời: *phương trình chuẩn tắc không phải một mẹo đại số, nó chỉ là cách viết lại câu “phần dư phải vuông góc với mọi thứ mô hình giải thích được”*. Công thức là hệ quả, hình học là nguyên nhân.

Ví dụ nhỏ nhất tính tay được. Khớp một hằng số c vào ba số đo 1, 2, 6. Ở đây $A = (1, 1, 1)^T$, không gian cột là đường chéo. Chiếu $b = (1, 2, 6)^T$ xuống đường ấy cho $c = (1 + 2 + 6)/3 = 3$, phần dư là $(-2, -1, 3)^T$, tổng bằng 0 — đúng là vuông góc với $(1, 1, 1)^T$. Trung bình cộng không phải một quy ước thống kê; nó là một phép chiếu. Hình 1 vẽ lại quan hệ đó ở dạng hai chiều.



Hình 1. Nghiệm bình phương tối thiểu là hình chiếu vuông góc của số đo b xuống không gian cột của A . Mọi biến thể — có trọng số, có ràng buộc, phi tuyến sau khi tuyến tính hoá — giữ nguyên bức tranh này và chỉ đổi cách đo góc vuông.

Bài toán phi tuyến trong SLAM không nằm ngoài bức tranh đó: Gauss–Newton tuyến tính hoá phần dư quanh điểm hiện tại, giải một bài chiếu, đi một bước, lặp lại. Ma trận $H = J^\top J$ mà bạn quen gọi là “Hessian xấp xỉ” chính là $A^\top A$ của bài chiếu ấy. Trong deep learning nó xuất hiện lại dưới tên *Gauss–Newton matrix*, và là bộ khung của mọi phương pháp bậc hai xấp xỉ. Đây là chỗ nối đầu tiên trong nhiều chỗ nối: cùng một ma trận, hai cộng đồng, hai cái tên.

I MẠCH 1 — từ phép chiếu tới các phân rã

- 1 Vấn đề** giải $A^\top Ax = A^\top b$ trực tiếp mất chính xác một cách khó hiểu.

Giải pháp không lập $A^\top A$ mà phân rã A bằng **QR** hoặc SVD rồi giải trên dạng đã phân rã.

Hệ quả trị kỳ dị của $A^\top A$ là bình phương trị kỳ dị của A , nên số điều kiện bị bình phương và bạn mất khoảng một nửa số chữ số có nghĩa.
- 2 Vấn đề** nhưng hiệu chỉnh chùm tia vẫn lập $J^\top J$ mỗi vòng.

Giải pháp bài toán được điều hoà bằng **giảm chấn** (*damping*) kiểu Levenberg–Marquardt, và cấu trúc thừa làm **Cholesky** rẻ hơn QR nhiều bậc.

Giá phải trả bạn đánh đổi độ chính xác số học lấy tốc độ, và trả giá đúng lúc bài toán suy biến — ví dụ chuỗi keyframe gần thẳng hàng.
- 3 Vấn đề** cần biết “hướng nào của bài toán yếu”.

Giải pháp phân rã trị riêng của ma trận đối xứng H , hoặc SVD của J .

Hệ quả trị riêng nhỏ nhất chỉ ra hướng mà dữ liệu gần như không ràng buộc: trong SLAM đó là *gauge freedom*, trong deep learning đó là hướng phẳng của **bề mặt hàm mất mát** (*loss landscape*) mà **hạ gradient** (*gradient descent*) đi rất chậm.

C. Ba phân rã, chọn cái nào

Ba công cụ hay bị dùng lẫn. Bảng 1 đặt chúng cạnh nhau; cột cuối là cột đáng giá nhất.

Bảng 1. Ba phân rã ma trận dùng nhiều nhất, so trên cùng tiêu chí. $m \times n$ với $m \geq n$.

Phân rã	Cơ chế	Chi phí	Mạnh khi nào	Hỏng / không dùng được khi
Cholesky	$H = LL^\top$ trên ma trận đối xứng xác định dương	$\frac{1}{3}n^3$, rất rẻ khi thưa	Bài toán lớn, thưa, đủ điều hoà — lựa chọn mặc định trong BA	H suy biến hoặc gần suy biến; đã bình phương số điều kiện từ trước
QR	Trực giao hoá cột của A	$\approx 2mn^2$	Bình phương tối thiểu cần độ chính xác, A điều hoà kém	Ma trận rất lớn; không cho biết phổ trị kỳ dị
SVD	$A = U\Sigma V^\top$, mọi ma trận	$\approx O(mn^2)$, đắt nhất	Cần biết hạng, cần xấp xỉ hạng thấp, cần chiếu về $SO(3)$	Kích thước lớn (phải dùng phương pháp lặp hoặc phác thảo ngẫu nhiên)

Quan hệ giữa phân rã trị riêng và SVD gọn trong một câu: trị kỳ dị của A là căn bậc hai của trị riêng của $A^\top A$, và vector kỳ dị phải là vector riêng của $A^\top A$. Phân rã trị riêng chỉ đẹp khi ma trận đối xứng (hiệp phương sai, Hessian, ma trận Gram của đặc trưng); SVD tồn tại cho mọi ma trận nên nó là lựa chọn mặc định khi ma trận là một phép biến đổi *giữa hai không gian khác nhau*, chẳng hạn ma trận trọng số của một tầng tuyến tính.

Hệ quả thực dụng dùng lại nhiều nhất là **định lý xấp xỉ hạng thấp**: cắt bỏ các trị kỳ dị nhỏ cho ra ma trận hạng k gần A nhất. Ba chỗ trong cây này sống trên nó:

- **Nén và phân tích biểu diễn — phân tích thành phần chính** (*principal component analysis, PCA*) chỉ là chiếu dữ liệu đã trừ trung bình lên k vector kỳ dị phải đầu tiên.
- **Ép về cấu trúc hợp lệ** — ước lượng ma trận cơ bản rồi ép trị kỳ dị thứ ba về không; chiếu một ma trận nhiều về $SO(3)$ bằng cách đặt cả ba trị kỳ dị bằng một. Đây là xấp xỉ hạng thấp có ràng buộc, và nó quay lại ở $A/01/04$ khi mạng phải xuất ra một phép quay.
- **Tinh chỉnh rẻ — LoRA** [3] giả thiết phần cập nhật trọng số khi tinh chỉnh có hạng thấp, nên chỉ học hai ma trận gầy thay vì cả ma trận đầy.

↔ **trường hợp riêng của C** — LoRA áp dụng định lý xấp xỉ hạng thấp, nhưng cho ma trận *cập nhật trọng số* thay vì cho ma trận dữ liệu

D. Số điều kiện: đại lượng dự báo rắc rối

Số điều kiện $\kappa(A) = \sigma_{\max}/\sigma_{\min}$ trả lời đúng một câu hỏi: nếu đầu vào sai lệch tương đối ε , nghiệm có thể sai lệch tương đối tới $\kappa\varepsilon$. Nói bằng lời: κ là hệ số khuếch đại sai số của bài toán, không phải của thuật toán.

Ví dụ tính tay. Với $A = \text{diag}(1; 0,01)$ thì $\kappa = 100$; một nhiễu 1% đặt đúng vào thành phần thứ hai của b làm thành phần thứ hai của nghiệm sai 100%. Ma trận không hề suy biến, định thức khác không, mọi phép tính vẫn chạy — và kết quả vẫn vô nghĩa. Đó là lý do định thức gần như vô dụng làm chỉ báo, còn κ thì không.

Trong deep learning cùng đại lượng ấy chi phối tốc độ hội tụ, và gần như mọi “mẹo huấn luyện” quen thuộc là một cách hạ κ gián tiếp:

- **Chuẩn hoá đầu vào và chuẩn hoá theo batch** (*batch normalization*) [4] — kéo các hướng về cùng thang đo, tức trực tiếp giảm tỉ số trị riêng lớn nhất trên nhỏ nhất.
- **Adam** [5] — chia mỗi toạ độ cho một ước lượng độ lớn riêng, tương đương đổi thang đo từng chiều, nên nó “thấy” một bài toán điều hoà tốt hơn bài toán thật.

- **Khởi tạo cẩn thận** — giữ phổ trị kỳ dị của từng tầng không quá lệch ngay từ đầu, để κ không tích lũy qua độ sâu.

↔ **hệ quả của D** — chuẩn hoá, khởi tạo và optimizer thích nghi ở chương huấn luyện đều là cách hạ số điều kiện một cách gián tiếp

Một câu để mang theo: *phần lớn cái gọi là mẹo huấn luyện là kỹ thuật điều hoà số học đội lốt*. Mạch này quay lại đầy đủ ở D khi bàn optimizer.

E. Giới hạn và trường hợp hỏng

Bốn công cụ trên có giới hạn rõ, và biết giới hạn quan trọng hơn biết công thức.

- **Số điều kiện phụ thuộc đơn vị đo.** Trong một bài BA trộn tịnh tiến tính bằng mét với góc quay tính bằng radian, đổi mét sang milimét làm κ đổi ba bậc mà bài toán không dễ hay khó đi. So sánh κ giữa hai mô hình tham số hoá khác nhau là so sánh vô nghĩa.
- **Ma trận suy biến không phải lúc nào cũng là bệnh.** Hessian của BA không ràng buộc gauge suy biến đúng bảy chiều — đó là cấu trúc thật của bài toán. Hessian của mạng nơ-ron cũng suy biến vì nhiều đối xứng nội tại: hoán vị neuron trong một tầng, co giãn bù trừ giữa hai tầng liên tiếp qua *ReLU*. Cách xử lý đúng là cố định gauge hoặc chấp nhận, không phải thêm nhiễu cho đủ hạng.
- **PCA giả định cấu trúc tuyến tính và đo bằng phương sai.** Đặc trưng của mạng sâu nằm trên một đa tạp cong. Vì thế “95% phương sai trong k chiều” không có nghĩa thông tin hữu ích nằm ở đó. Hướng phương sai lớn thường là độ sáng toàn cục hoặc phong cách ảnh; hướng quyết định nhân có thể nằm ở phương sai nhỏ.

↔ **tiền đề E** — *linear probe* là bài bình phương tối thiểu trên không gian đặc trưng, nên chất lượng của nó bị chi phối bởi chính số điều kiện ở đây

Vì thế trực quan hoá đặc trưng hiện đại dùng *t-SNE* [6] hay *UMAP* [7]. Nhưng hai thứ đó đánh đổi tính trung thực về khoảng cách toàn cục — một trade-off mỗ ở E.

- **Chi phí.** SVD đầy đủ của ma trận $50\,000 \times 2048$ là chuyện của phương pháp lập hoặc phác thảo ngẫu nhiên, không phải một lời gọi thư viện vô tư.

CẠM BÃY

Đừng đọc “hạng thấp” như “ít thông tin”. Ma trận đặc trưng của một mô hình tự giám sát bị *rank collapse* là bệnh; ma trận cập nhật LoRA có hạng thấp là giả thiết thiết kế có chủ đích. Cùng một con số hạng, hai ý nghĩa trái ngược, và cái phân biệt chúng là *ma trận đó đang biểu diễn dữ liệu hay biểu diễn một phép hiệu chỉnh*. Trong thực hành nên đo **hạng hiệu dụng**, tức entropy của phổ trị kỳ dị đã chuẩn hoá. Đừng đếm số trị kỳ dị khác không: với số dấu phẩy động thì hầu như không trị kỳ dị nào đúng bằng không.

CẦN NHỚ

Bình phương tối thiểu là một phép chiếu vuông góc. Phân rã là danh sách trực kèm hệ số kéo dẫn. Số điều kiện nói bạn sắp mất bao nhiêu chữ số có nghĩa — và trong huấn luyện, nó nói bạn sắp chờ lâu cỡ nào. Ba câu đó phủ gần hết chỗ đại số tuyến tính cần cho cả cây này. [T1]

F. Kết nối

Mục này là tiền đề trực tiếp của A/01/02-vi-phan-va-toi-uu: hình dạng landscape bàn ở đó chính là phổ trị riêng của Hessian nói ở đây. Nó cũng là tiền đề của A/01/04-hinh-hoc-va-tin-hieu, nơi $SO(3)$ xuất hiện như một ràng buộc trực giao mà SVD dùng để ép về. Theo chiều ngang, C dùng lại xấp xỉ hạng thấp ở phần tinh chỉnh hiệu quả tham số. Còn E dùng lại phép chiếu khi bàn *linear probe*. Một linear probe *chính là* bài bình phương tối thiểu trên không gian đặc trưng, nên nó thừa hưởng đúng số điều kiện đã bàn ở trên. Mini project cuối chương (PCA và *whitening* trên CIFAR-10) là bài kiểm tra thực hành cho toàn mục.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao giải phương trình chuẩn tắc bằng cách lập $A^T A$ làm mất khoảng một nửa số chữ số có nghĩa, trong khi QR thì không?
2. Số điều kiện lớn của một ma trận trọng số nói gì và *không* nói gì về độ ổn định huấn luyện?
3. Hessian của hiệu chỉnh chùm tia suy biến bảy chiều. Vì sao đó không phải lỗi, và làm gì thì đúng?
4. Bạn đo được 95% phương sai của đặc trưng nằm trong 10 chiều đầu. Vì sao điều đó không cho phép kết luận có thể nén đặc trưng về 10 chiều mà không mất độ chính xác?
5. Cùng là “hạng thấp”, vì sao rank collapse trong học tự giám sát là bệnh còn LoRA lại là tính năng?
6. Bạn phải chọn giữa Cholesky, QR và SVD cho một bài bình phương tối thiểu 20 nghìn phương trình, 300 ẩn, ma trận thưa và điều hoà tốt. Chọn cái nào và vì sao?

Nguồn

- [1] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*. SIAM, 1997.
- [2] M. P. Deisenroth, A. A. Faisal, and C. S. Ong, *Mathematics for Machine Learning*. Cambridge University Press, 2020.
- [3] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [4] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [5] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [6] L. van der Maaten and G. Hinton, “Visualizing Data Using t-SNE,” *JMLR*, vol. 9, pp. 2579–2605, 2008.
- [7] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.